FIREBRAND
LYCEUM EDUCATION GROUP

Module 4: Software Design
Session 4 stretch: structure and flow

Software Developer Level 4



Coupling and cohesion

The idea

Coupling is how much one module depends on another. Cohesion is how closely the parts inside one module serve one purpose.

Stevens, Myers and Constantine named both in 1974: fewer, simpler connections between modules make each one easier to understand and change.

Why it matters for a developer

Low coupling and high cohesion keep a change in one place, so fixes do not ripple through the system.

Try this

Pick one class in your own project. Describe it in one sentence, then count what else changes if it changes. Keep it general: no employer code or confidential detail.

FIREBRAND
LYCEUM EDUCATION GROUP

The C4 model

Four zoom levels, so each reader gets the detail they need, like a map.

1. **Context**
   the portal, its users and the systems it talks to

2. **Containers**
   the apps and data stores inside the portal

3. **Components**
   the main parts inside one app

4. **Code**
   classes, often a UML class diagram, only where it helps

> **Try this: draw the portal's context diagram in six boxes or fewer.**

FIREBRAND
LYCEUM EDUCATION GROUP

Little's law

JOHN LITTLE, 1961

> **Average work waiting equals the arrival rate times the average wait.**

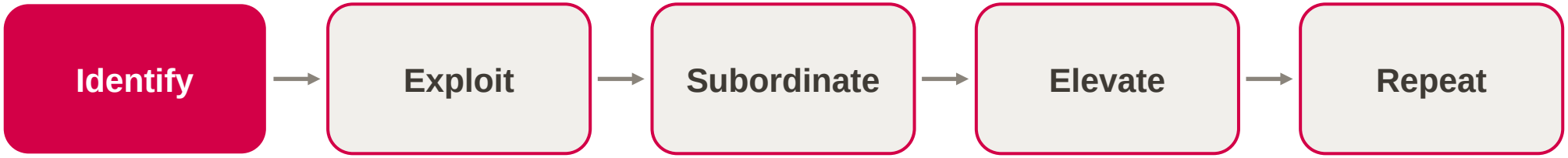
Why it matters: it turns a stage time into a queue a manager can picture, and shows that a shorter wait means a smaller pile.

Try this: take the channel you found in Activity 3. Kestrel Moor gets about 21,000 requests a year; using that channel's share of the sample and its mean wait before triage, how many of its requests wait untriaged on an average day?

FIREBRAND LYCEUM EDUCATION GROUP

The theory of constraints

A process goes only as fast as its tightest step, so improve that step first.



find the limit — get more from it — pace the rest to it — then add capacity — start again

Why it matters: speeding up any other step only builds a bigger queue in front of the constraint.

Try this: take the bottleneck you found in Activity 3. Which step does your improvement belong to?

The dependency rule

Source code dependencies should point only inward, towards the business rules.

CLASSIC LAYERS

The rules depend on the database code.

Application calls down to data.

Change the database and the rules may need to change too.

CLEAN ARCHITECTURE

The database code depends on the rules.

Application defines an interface; data implements it.

Try this: in your Activity 1 diagram, which way does the arrow between Application and Data point?

FIREBRAND
LYCEUM EDUCATION GROUP



Optional self-study, about 30 minutes.

This deck is for you if you want the ideas behind today's session. There are five slides, one idea each: coupling and cohesion, the C4 model, Little's law, the theory of constraints and the dependency rule. Each slide gives the idea in plain words, why it matters for a developer and a 'Try this' question to apply it to Kestrel Moor Homes or to your own project. These notes give the full reference for each idea and say where an idea is debated.

Work through it alone, or bring your answers to the start of Session 5. Keep any example from your own work general: no personal or customer data, credentials or confidential material, and no employer data in AI tools your employer has not approved. Conway's law was covered in Session 1, so it is not repeated here.

Coupling and cohesion

The idea

Coupling is how much one module depends on another. Cohesion is how closely the parts inside one module serve one purpose.

Stevens, Myers and Constantine named both in 1974: fewer, simpler connections between modules make each one easier to understand and change.

Why it matters for a developer

Low coupling and high cohesion keep a change in one place, so fixes do not ripple through the system.

Try this

Pick one class in your own project. Describe it in one sentence, then count what else changes if it changes. Keep it general: no employer code or confidential detail.

FIREBRAND
LYCEUM EDUCATION GROUP

2

The idea: coupling measures how much one part of a system depends on the inside of another; cohesion measures how closely the things inside one part serve a single purpose. The paper that named them argues that fewer and simpler connections between modules make each module easier to understand, and limit how far a change spreads.

Why it matters: these are the quickest tests you can apply to any class. If you cannot describe a class in one sentence without 'and', its cohesion is low; if changing it forces changes in many other classes, its coupling is tight.

Try this: pick one class in your own project, describe it in one sentence, then count what else would change if it changed. A good answer names the class, gives its purpose without 'and' and counts its dependants honestly. If the count is high, say what narrow interface would reduce it; if the sentence needs 'and', say how you would split the class.

Debate: the ideas are widely accepted, but the many proposed coupling and cohesion metrics are debated and none is a universal standard.

Reference: W. P. Stevens, G. J. Myers and L. L. Constantine, 'Structured design', IBM Systems Journal, 13(2), 1974, pages 115 to 139.

The C4 model

Four zoom levels, so each reader gets the detail they need, like a map.

1. **Context**
   the portal, its users and the systems it talks to
2. **Containers**
   the apps and data stores inside the portal
3. **Components**
   the main parts inside one app
4. **Code**
   classes, often a UML class diagram, only where it helps

Try this: draw the portal's context diagram in six boxes or fewer.

FIREBRAND LYCEUM EDUCATION GROUP

3

The idea: C4 draws software architecture at four zoom levels, like a map. A system context diagram shows the system, the people who use it and the systems it talks to; a container diagram shows the applications and data stores inside it; a component diagram shows the main parts inside one application; and the code level, often a UML class diagram, is drawn only where it helps.

Why it matters: it links today's two themes, structure and audience. A context diagram suits a manager or a tenant panel; the component and code levels suit developers. Today's class diagram is a code-level view.

Try this: draw the Tenant Repairs Portal's context diagram in six boxes or fewer. A good answer puts the portal in the centre, the people who use it around it (tenants, contact centre staff, operatives and contractors) and the systems it talks to (from Case Brief, Part 1), with every arrow labelled with what passes along it.

Debate: C4 is a convention, not a formal standard. It does not prescribe a notation, and many teams use UML or other notations instead; choose what your readers can follow.

Reference: Simon Brown, 'The C4 model for visualising software architecture', c4model.com.

Little's law

JOHN LITTLE, 1961

> **Average work waiting equals the arrival rate times the average wait.**

Why it matters: it turns a stage time into a queue a manager can picture, and shows that a shorter wait means a smaller pile.

Try this: take the channel you found in Activity 3. Kestrel Moor gets about 21,000 requests a year; using that channel's share of the sample and its mean wait before triage, how many of its requests wait untriaged on an average day?

FIREBRAND LYCEUM EDUCATION GROUP

4

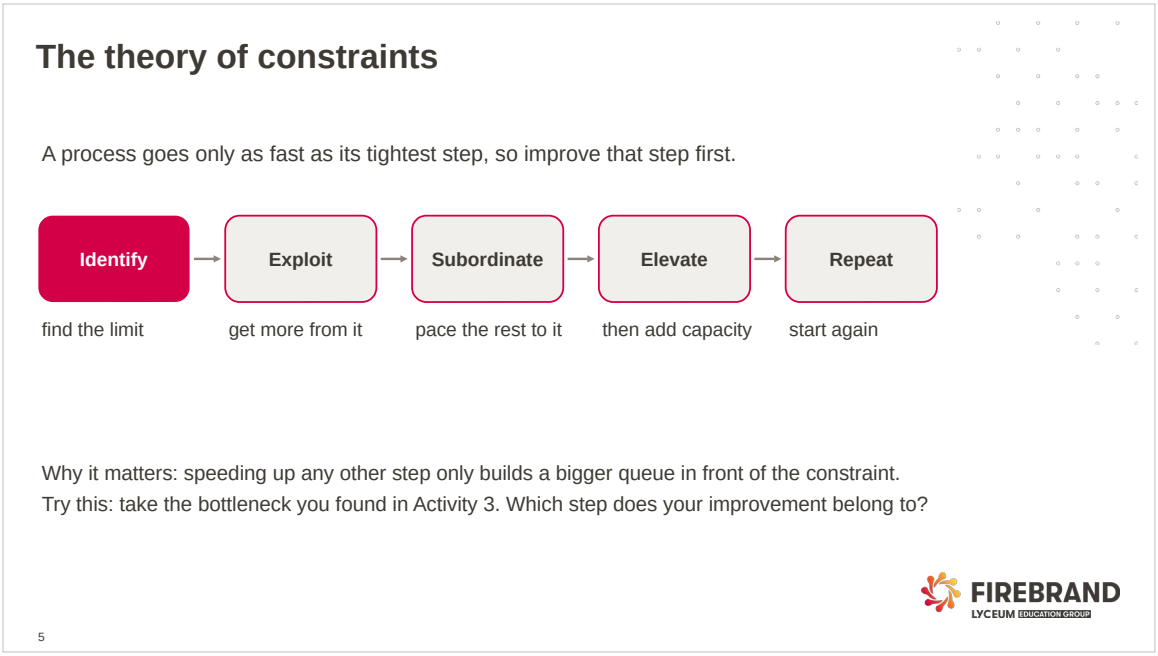
The idea: in a stable system, the average number of items in it (L) equals the rate at which items arrive (lambda) times the average time each item spends in it (W): L = lambda x W.

Why it matters: it turns a stage time into a pile of work a manager can picture. Halve the wait and, at the same arrival rate, the pile halves.

Try this: take the channel you found in Activity 3. Work out its share of the 150 requests in the sample, multiply by about 21,000 requests a year (Case Brief, Part 1) and divide by 365 to get requests a day. Turn that channel's mean wait before triage into days by dividing by 24. Multiply the two. Worked method on different numbers: a queue that receives 10 items a day, each waiting half a day on average, holds about 10 x 0.5 = 5 items at any time. Then say what limits your answer: six weeks of 150 requests, a channel mix that may change through the year, and calendar hours that include nights and weekends.

Debate: none about the result itself. It is a proven theorem, provided the system is stable over the period you measure.

Reference: John D. C. Little, 'A Proof for the Queuing Formula: L = lambda W', Operations Research, 9(3), 1961, pages 383 to 387.



The idea: a process can go only as fast as its tightest step, the constraint. Goldratt's five focusing steps are: identify the constraint; exploit it (get the most from it as it is); subordinate everything else to it (pace the other steps to it); elevate it (add capacity); then repeat, so that inertia does not become the new constraint.

Why it matters for developers: improving a step that is not the constraint looks good locally but does not help the whole flow; it just builds a bigger queue in front of the constraint.

Try this: take the bottleneck you found in Activity 3 and the improvement you proposed. Which of the five steps does it belong to? A good answer names the step and gives a reason. Removing work that wastes the constraint's time, such as re-typing, is exploiting it; making every other step feed it at its pace is subordinating; adding staff or capacity is elevating, which Goldratt places after the first steps.

Debate: the theory of constraints comes from manufacturing. How well it transfers to knowledge work and software teams, where work items vary in size and the constraint moves often, is debated, so treat it as a useful lens rather than a law.

Reference: Eliyahu M. Goldratt and Jeff Cox, 'The Goal: A Process of Ongoing Improvement', North River Press, 1984.

The dependency rule

Source code dependencies should point only inward, towards the business rules.

CLASSIC LAYERS

The rules depend on the database code.

Application calls down to data.

Change the database and the rules may need to change too.

CLEAN ARCHITECTURE

The database code depends on the rules.

Application defines an interface; data implements it.

Try this: in your Activity 1 diagram, which way does the arrow between Application and Data point?

FIREBRAND LYCEUM EDUCATION GROUP

6

The idea: Robert C. Martin's dependency rule says that source code dependencies can only point inwards, towards the business rules. Nothing in an inner circle (entities and use cases) may name anything in an outer circle (databases, frameworks, the web).

Why it matters: in classic layering the application code imports the data layer, so a change of database can reach the rules. With the dependency rule, the application defines an interface (for example a repository it needs) and the data layer implements it, so the rules never import database code. The call still goes down at run time; only the code dependency is reversed. This is the dependency inversion principle.

Try this: in your Activity 1 diagram, which way does the arrow between Application and Data point? Then sketch the alternative: the service depends on an interface it owns, and a class in the data and integration layer implements it (realisation: a dashed line with a hollow triangle). Say which outside system you would most want to protect this way, and why.

Debate: critics argue that the extra interfaces add cost with little benefit for small or short-lived systems; supporters say the benefit grows with the system's life. Both views are reasonable; judge by how often the outer parts are likely to change.

Reference: Robert C. Martin, 'The Clean Architecture', The Clean Code Blog, 13 August 2012, and 'Clean Architecture: A Craftsman's Guide to Software Structure and Design', Pearson, 2017.